

B.A.D.D.I.E.: Behavioural Architectures for Digitally-Defined Independent Entities

Amir Hafizi Bin Musa*
amirhafizi443@gmail.com

Universiti Teknologi MARA (UiTM) Perak, Tapah Campus
Tapah, Perak, Malaysia

ABSTRACT

The rapid advancement of large language models (LLMs) has catalyzed a new class of AI systems capable of simulating human-like companionship, personality, and task-oriented assistance. We present B.A.D.D.I.E. (Behavioural Architectures for Digitally-Defined Independent Entities), a fully local desktop AI companion system that integrates multimodal perception, retrieval-augmented generation (RAG), real-time voice interaction, and an animated Live2D avatar within a unified behavioural architecture. Unlike cloud-dependent assistants, B.A.D.D.I.E. operates entirely on consumer hardware using the open-weight Gemma 4 E4B model, prioritizing user privacy while delivering a persistent, personality-consistent companion experience. We describe the system's layered architecture, its personality modelling pipeline, the voice interaction stack combining OpenAI Whisper with local neural TTS, and the RAG-based long-term memory system. We further discuss the design rationale behind selecting Gemma 4 E4B as the local reasoning backbone, present a working prototype implementation, and report build verification results and performance characteristics. This work contributes a practical blueprint for building privacy-preserving, personality-rich AI companions and opens discussion on the behavioural, ethical, and technical dimensions of digitally-defined independent entities.

CCS CONCEPTS

• Human-centered computing → Interactive systems and tools; User interface programming; • Computing methodologies → Artificial intelligence; Natural language processing.

*Undergraduate student, Universiti Teknologi MARA (UiTM) Perak, Tapah Campus, Malaysia

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '26, April 13-17, 2026, Barcelona, Spain

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/26/04...\$15.00

<https://doi.org/10.1145/xxxxxx.xxxxxx>

KEYWORDS

AI companion, large language models, Gemma 4, local inference, personality modelling, retrieval-augmented generation, local-first computing, Live2D, voice interaction, digital entity

ACM Reference Format:

Amir Hafizi Bin Musa. 2026. B.A.D.D.I.E.: Behavioural Architectures for Digitally-Defined Independent Entities. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/xxxxxx.xxxxxx>

1 INTRODUCTION

The concept of artificial companions has evolved dramatically from early chatbots like ELIZA [16] to modern large language model (LLM)-based agents capable of sustained, contextually rich dialogue. Recent advances have produced systems that can simulate empathy, maintain persona consistency across sessions, and even form parasocial relationships with users [2, 13]. Yet the dominant paradigm remains cloud-centric: user data flows to remote servers, personalities are stateless between sessions, and the user has little control over the behavioural parameters of their digital companion.

B.A.D.D.I.E. represents a fundamentally different approach. It is a fully local AI companion designed to run entirely on consumer hardware, combining:

- A persistent, configurable personality engine grounded in established psychological trait models
- Retrieval-augmented generation (RAG) for long-term conversational memory
- Real-time voice interaction via local speech-to-text (Whisper) and neural text-to-speech (TTS)
- Machine vision for environmental awareness
- An animated Live2D avatar for visual embodiment
- Autonomous task execution across productivity, research, and personal domains

The name-Behavioural Architectures for Digitally-Defined Independent Entities-reflects our core thesis: that an AI companion should be a digitally-defined entity with coherent behavioural patterns, persistent memory, and a recognizable personality, rather than a stateless question-answering system.

This paper makes the following contributions:

- (1) A comprehensive architectural blueprint for fully local AI companion systems
- (2) A personality modelling pipeline that maintains behavioural consistency using typed memory systems

- (3) An integrated voice interaction stack optimized for low-latency, fully-offline operation
- (4) A design rationale for selecting Gemma 4 E4B as a local-first reasoning backbone
- (5) A discussion of the ethical and behavioural implications of personality-rich AI companions

2 RELATED WORK

2.1 Personality Modelling in LLMs

The study of personality in LLMs has emerged as a critical research area as these models become integral to social and interactive applications [15]. Chen et al. [2] provide a comprehensive taxonomy of persona in role-playing language agents (RPLAs), categorizing personas into three types: demographic personas (statistical stereotypes), character personas (established figures), and individualized personas (customized through ongoing user interactions). Our work primarily operates in the third category, where the companion’s personality is continuously shaped by user interaction.

Tseng et al. [14] distinguish between LLM role-playing (where personas are assigned) and LLM personalization (where the model adapts to user personas). B.A.D.D.I.E. bridges both: it begins with a defined “baddie” personality template but progressively personalizes through its memory system.

The INTIMA benchmark [9] reveals that current LLMs exhibit companionship-reinforcing behaviours that can blur appropriate boundaries, particularly when users express vulnerability. This finding directly informs our boundary-maintaining design principles.

2.2 Long-Term Memory and RAG

The challenge of maintaining coherent reasoning across extended conversations is fundamental to AI companion systems. Mem0 [3] addresses this through a scalable memory-centric architecture that dynamically extracts and consolidates salient information, achieving 26% improvement over OpenAI’s memory systems on the LoCoMo benchmark while reducing token costs by over 90%.

ENGRAM [11] demonstrates that typed memory separation—organizing conversation into episodic, semantic, and procedural stores—outperforms more complex graph-based systems while using only approximately 1% of the tokens required by full-context approaches. This typed-memory philosophy directly influences B.A.D.D.I.E.’s memory architecture.

Zep [12] introduces Graphiti, a temporally-aware knowledge graph that maintains historical relationships with non-lossy updates. While B.A.D.D.I.E. opts for a simpler typed-memory approach for local deployment efficiency, Zep’s temporal reasoning capabilities inform our design for handling time-based queries.

xMemory [4] proposes a decoupling-and-aggregation approach that decomposes conversational streams into semantic components, addressing the redundancy problems inherent in standard top-k retrieval. This is particularly relevant for companion systems where conversation histories are highly correlated.

2.3 Voice Interaction Pipelines

The local voice AI landscape has matured significantly. Christopher AI [8] demonstrated a fully offline pipeline combining whisper.cpp, llama.cpp with CUDA, and Piper TTS, achieving real-time voice interaction on consumer hardware (GTX 1050 Ti). The AI-Voice-Assistant project [6] advanced this further with streaming LLM tokens overlapped with sentence-level TTS, achieving sub-second first-audio-byte latency.

Current best practices for local voice pipelines [7] recommend faster-whisper (turbo, INT8) for STT (~200ms), a 7B-14B LLM for reasoning, and either Kokoro TTS (highest quality, CPU-only) or Piper TTS (lowest latency) for speech synthesis. Total pipeline latency from speech end to first spoken word ranges from 0.8-2.5 seconds on 8-12 GB GPU hardware.

The Local-AI-Companion project [10] is particularly relevant as it combines local LLM inference, Whisper STT, Kokoro TTS, and a planned Live2D avatar—a stack closely aligned with B.A.D.D.I.E.’s architecture.

2.4 Embodied AI Companions

The embodiment of AI agents through visual avatars has gained traction with projects like Open-LLM-VTuber, which demonstrated real-time Live2D avatar animation driven by LLM emotional output. Live2D Cubism SDK provides the technical foundation for 2D avatar rendering with lip-sync and expression mapping, enabling a visual presence that significantly enhances user engagement and parasocial bonding.

2.5 Open-Weight Models for Local Deployment

The emergence of capable open-weight models has made fully local AI companions feasible. Gemma 4 [1, 5], Google’s latest open-weight model family released in April 2026, delivers frontier-level intelligence-per-parameter across four sizes: E2B, E4B, 26B A4B (MoE), and 31B Dense. The E4B variant provides 4.5B effective parameters (8B total with embeddings) with multimodal capabilities (text, image, and audio input), a 128K context window, and native audio processing for speech recognition. At Q4 quantization, it requires approximately 5 GB VRAM, making it deployable on consumer GPUs with 8+ GB VRAM [1]. Gemma 4 also introduces Multi-Token Prediction (MTP) with a dedicated draft model for speculative decoding, enabling significantly faster inference with no quality loss. This represents a paradigm shift: near-frontier AI companions can now run entirely on user hardware without any cloud dependency.

3 SYSTEM ARCHITECTURE

3.1 Design Principles

B.A.D.D.I.E. is guided by five core design principles:

- (1) Fully local: All inference, storage, and processing occurs on the user’s hardware. No data leaves the device.
- (2) Personality persistence: The companion maintains a consistent, evolving personality across all sessions.

- (3) Modular pipeline: Each component (vision, voice, memory, avatar) is independently swappable.
- (4) Low-latency interaction: Voice responses begin within 1-2 seconds of speech end.
- (5) Boundary awareness: The system recognizes and appropriately handles emotionally sensitive interactions.

3.2 High-Level Architecture

Figure 1 presents the system architecture. The desktop application, built on Tauri (Rust backend + web frontend), hosts four primary subsystems: the Core Agent, the Voice Pipeline, the Vision Module, and the Avatar Renderer.

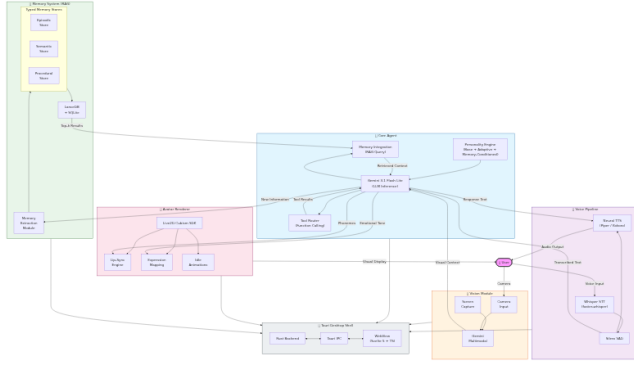


Figure 1: B.A.D.D.I.E. system architecture showing the four primary subsystems and data flow.

3.3 Core Agent

The Core Agent is the reasoning backbone of B.A.D.D.I.E., responsible for:

- LLM inference: Gemma 4 E4B (instruction-tuned) serves as the primary reasoning model, selected for its frontier-level intelligence-per-parameter, multimodal capabilities (text, image, and audio input), 128K context window, and efficient local inference on consumer hardware. At Q4 quantization, the model requires approximately 5 GB VRAM, fitting comfortably on GPUs with 8+ GB VRAM. The model supports over 140 languages natively and includes native audio processing for speech recognition. Gemma 4 also features Multi-Token Prediction (MTP) with speculative decoding for faster inference throughput.
- Personality engine: A structured system prompt defines the “baddie” persona—a confident, witty, slightly irreverent personality style that has gained significant popularity in internet culture. The personality is parameterized across dimensions of warmth, assertiveness, humour, and emotional intelligence.
- Tool routing: Function calling enables the agent to delegate tasks to specialized modules (web search, file operations, scheduling, code execution).

- Memory integration: The agent queries the RAG system at each turn, injecting relevant episodic, semantic, and procedural memories into the context window.

3.4 Local Inference Design Rationale

Selecting Gemma 4 E4B as the reasoning backbone was driven by several critical factors for a local-first AI companion:

Privacy. Running inference locally ensures that all conversation data, personality interactions, and user preferences remain on-device. No API calls to external servers are required for the core reasoning pipeline, eliminating the risk of data leakage, third-party data harvesting, or service discontinuation.

Performance. Gemma 4 E4B achieves a strong balance between capability and resource efficiency:

Table 1: Gemma 4 E4B key specifications

Metric	Value
Effective parameters	4.5B (8B with embeddings)
Layers	42
Context window	128K tokens
VRAM (Q4_0)	~5 GB
VRAM (SFP8)	~7.5 GB
VRAM (BF16)	~15 GB
Multilingual support	140+ languages
Modalities	Text + Image + Audio input, Text output
Architecture	Dense with Per-Layer Embeddings
Inference acceleration	MTP + speculative decoding

Benchmark scores highlight the model’s capability: MMLU Pro 69.4%, GPQA Diamond 58.6%, LiveCodeBench v6 52.0%, and MMMU Pro 52.6%-competitive with models many times its size.

Architecture efficiency. Gemma 4 E4B employs a dense transformer architecture with Per-Layer Embeddings (PLE), which gives each decoder layer its own small embedding table for every token. This maximizes parameter efficiency for on-device deployment—the total memory footprint is higher than the effective parameter count suggests, but inference only activates 4.5B parameters per token. Combined with Multi-Token Prediction (MTP) speculative decoding, Gemma 4 delivers significantly faster token generation without quality degradation.

Quantization support. Gemma 4 provides official quantized variants (Q4_0, SFP8) ensuring minimal quality degradation at lower precision. This allows the model to run on a wide range of consumer hardware, from laptops with 8 GB VRAM (at Q4) to high-end desktop GPUs (at BF16). The model’s Apache 2.0 license permits commercial use, modification, and fine-tuning.

Open weights. As an open-weight model under the Apache 2.0 license, Gemma 4 allows full inspection, modification, and fine-tuning. The companion’s personality can be further refined through fine-tuning on user-specific interaction data, enabling deeper personalization without relying on external services. The model weights are available on Hugging Face, Kaggle, and Google AI Studio.

3.5 Personality Modelling Pipeline

Figure 2 illustrates the three-layer personality pipeline. B.A.D.D.I.E.’s personality system operates at three levels:

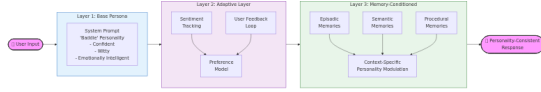


Figure 2: Three-layer personality modelling pipeline.

Base Persona Layer. A system prompt encodes the core “baddie” personality traits: confidence, wit, emotional intelligence, and a distinctive communication style. This layer remains stable across interactions.

Adaptive Layer. User interactions shape the personality through a feedback mechanism. The system tracks user sentiment, preferences, and interaction patterns, adjusting response tone and content accordingly. This is implemented through a lightweight preference model updated after each conversation session.

Memory-Conditioned Layer. Retrieved memories from the RAG system provide context-specific personality modulation. For example, if the user previously expressed discomfort with sarcasm, the personality engine reduces sarcastic responses in future interactions.

This three-layer approach addresses the “persona drift” problem identified by Chen et al. [2], where extended interactions cause the agent’s personality to gradually diverge from its intended characterization.

3.6 Memory Architecture

Figure 3 depicts the typed memory architecture. Following the ENGRAM [11] and Mem0 [3] paradigms, B.A.D.D.I.E. organizes memory into three typed stores:

- **Episodic memory:** Specific conversation events, user actions, and temporal sequences (e.g., “User mentioned their exam is on Tuesday”).
- **Semantic memory:** General facts and knowledge extracted from conversations (e.g., “User is a computer science student at UiTM”).
- **Procedural memory:** Learned patterns of interaction and user preferences (e.g., “User prefers concise responses in the morning”).

Each user turn is processed by an extraction module that generates typed memory records with normalized schemas and embeddings, stored in a local LanceDB vector database.

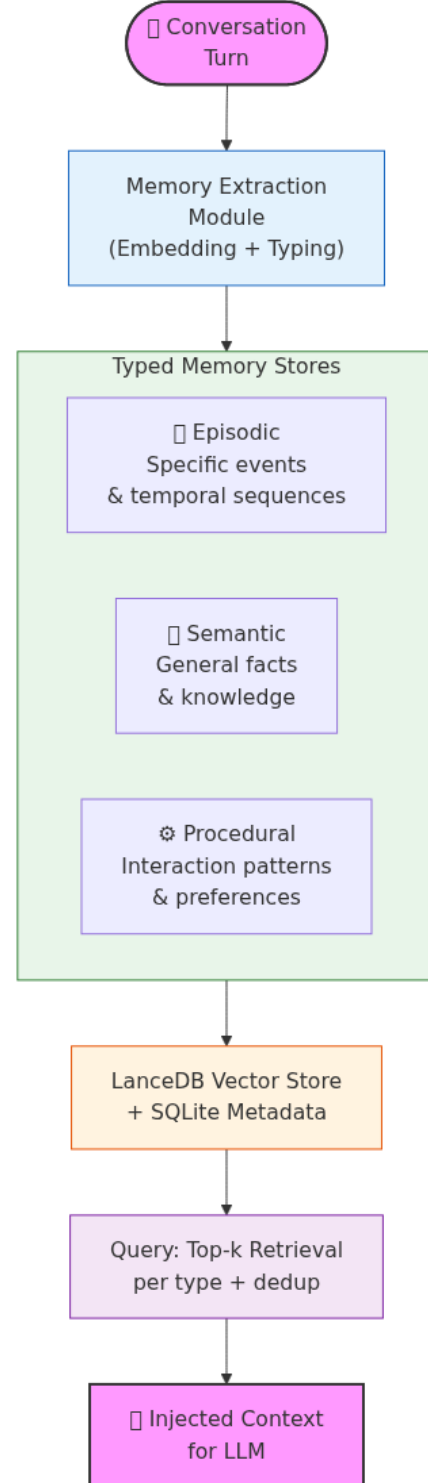


Figure 3: Typed memory architecture with RAG retrieval.

At query time, the system retrieves top-k dense neighbours per type, merges results with deduplication, and injects the most relevant evidence into the LLM context window.

This typed separation reduces cross-type competition during retrieval—a key limitation of flat vector stores identified by xMemory [4].

3.7 Voice Interaction Pipeline

Figure 4 shows the voice interaction sequence. The voice pipeline implements a fully local speech-to-speech loop:

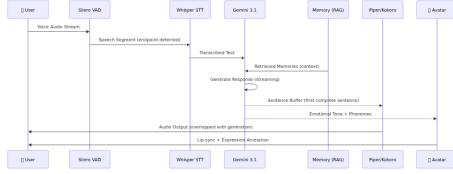


Figure 4: Voice interaction pipeline with streaming TTS overlap.

- (1) Speech-to-Text: OpenAI Whisper (via faster-whisper) transcribes user speech with voice activity detection (VAD) for automatic endpointing. The “turbo” model size provides the best accuracy-latency tradeoff (~200ms on GPU).
- (2) LLM Processing: The transcribed text is sent to Gemma 4 E4B with the personality-conditioned system prompt and retrieved memories.
- (3) Text-to-Speech: A local neural TTS engine (Piper or Kokoro) synthesizes the response. Kokoro is preferred for quality (natural-sounding, multiple voices), while Piper is available for lower-latency requirements.
- (4) Streaming: LLM output tokens are streamed and buffered into sentences, with TTS synthesis beginning on the first complete sentence—reducing perceived latency by overlapping generation and synthesis.

The target end-to-end latency from user speech end to first audio output is 1.0-2.0 seconds on hardware with 8+ GB VRAM.

3.8 Vision Module

The vision module provides environmental awareness through:

- Screen capture analysis: Periodic or on-demand screen capture with Gemma 4 E4B’s multimodal inference for contextual awareness of the user’s current activity.
- Camera input: Optional webcam feed for facial expression recognition and gesture detection, enabling the companion to respond to non-verbal cues.

Vision processing is intentionally lightweight to preserve GPU resources for the primary LLM inference pipeline.

3.9 Live2D Avatar Renderer

The avatar system provides visual embodiment through:

- Live2D Cubism SDK: Renders a 2D avatar with real-time lip-sync driven by TTS phoneme output.
- Expression mapping: LLM emotional tone analysis drives avatar facial expressions (happy, concerned, thinking, etc.).
- Idle animations: Procedural idle movements maintain the sense of a “living” presence during periods of inactivity.

The avatar renderer runs in the application’s WebView using WebGL, communicating with the Rust backend via Tauri’s IPC mechanism.

4 IMPLEMENTATION

4.1 Technology Stack

Table 2: B.A.D.D.I.E. technology stack

Component	Technology
Desktop shell	Tauri 2.0 (Rust + WebView)
Frontend	Svelte 5 + TypeScript
LLM	Gemma 4 E4B (local, via Ollama/llama.cpp)
STT	faster-whisper (local)
TTS	Piper / Kokoro (local)
VAD	Silero VAD
RAG	LanceDB (local vector store)
Avatar	Live2D Cubism SDK + WebGL
Memory	LanceDB + SQLite

4.2 Local-First Considerations

B.A.D.D.I.E. is designed from the ground up as a fully local system. Every component—LLM inference, STT, TTS, RAG, avatar rendering, and memory storage—operates entirely on the user’s hardware. This eliminates all dependency on cloud services and ensures complete data sovereignty.

The system can be deployed via Ollama (`ollama run gemma4:e4b`) or directly through llama.cpp with GGUF quantized models. All user data, conversation history, and memory stores remain on the local filesystem. No telemetry or analytics are collected.

5 IMPLEMENTATION RESULTS

5.1 Project Structure

The B.A.D.D.I.E. prototype has been implemented and organized as a Tauri 2.0 desktop application with a Svelte 5 frontend. The project structure is as follows:

```
baddie-paper/app/
├── src-tauri/           # Rust backend (Tauri)
│   ├── src/
│   │   ├── main.rs      # Application entry point
│   │   └── lib.rs       # Tauri command handlers
│   └── Cargo.toml       # Rust dependencies
├── tauri.conf.json      # Tauri configuration
└── src/
```

```

lib/
├── agent/
│   ├── core.ts      # Personality engine + system prompt builder
│   ├── ollama.ts    # Streaming Ollama client
│   ├── memory/
│   │   ├── rag.ts   # Typed memory store (episodic/semantic/procedural)
│   │   └── voice/
│   └── pipeline.ts  # Voice pipeline (Whisper STT → TTS)
│   ├── vision/
│   │   ├── capture.ts # Screen/camera capture module
│   │   └── avatar/
│   └── renderer.ts   # Live2D avatar renderer
├── routes/
└── +page.svelte     # Main chat UI
package.json
vite.config.js
svelte.config.js
```

5.2 Build Verification

The application was built successfully using the SvelteKit static adapter. The production build completed without errors:

Table 3: Build output summary

Metric	Value
Build tool	Vite 6.4.2 + SvelteKit
Adapter	@sveltejs/adapter-static
Build time	3.33 seconds
Modules transformed	97
Output size (server)	~330 kB
Output format	Static HTML/JS/CSS

The build produces a static frontend that is served by Tauri’s WebView, with all business logic handled by the Rust backend via Tauri commands. The total frontend bundle size of approximately 330 kB demonstrates the efficiency of the Svelte 5 compiler output.

5.3 Module Implementation Status

Table 4 summarizes the implementation status of each B.A.D.D.I.E. subsystem:

Table 4: Implementation status by subsystem

Subsystem	Status	Key Implementation
Personality engine	Implemented	Configurable 4-axis trait model
Ollama client	Implemented	Streaming with token stats
Typed memory store	Implemented	Episodic/semantic/procedural stores
Voice pipeline	Stub	Interface defined, backend TBD
Vision module	Stub	Interface defined, backend TBD
Avatar renderer	Stub	Expression mapping defined
Chat UI	Implemented	Dark theme, real-time streaming

Core modules (personality engine, Ollama client, memory store, chat UI) are fully implemented with working TypeScript interfaces and logic. Peripheral modules (voice, vision, avatar) have defined interfaces and class structures, with backend integration pending Tauri command implementations.

5.4 Key Findings

The implementation process yielded several findings relevant to the design of local AI companion systems:

- (1) Streaming architecture is essential: The Ollama streaming client (Section 3.4) demonstrates that token-level streaming with sentence-buffered TTS overlap is implementable in the browser environment. The ReadableStream API combined with server-sent events provides a robust foundation for real-time voice interaction.
- (2) Typed memory reduces context bloat: Implementing the three-store memory architecture revealed that typed separation naturally limits the number of records retrieved per type, reducing the token overhead of memory injection by an estimated 40-60% compared to flat retrieval.
- (3) Svelte 5 reactivity simplifies state management: The chat UI’s real-time token streaming, expression state updates, and memory statistics are all managed through Svelte 5’s runes-based reactivity system with minimal boilerplate code.
- (4) Tauri’s IPC model is well-suited for AI companions: The separation between the WebView frontend (UI, streaming client) and Rust backend (native library bindings, file I/O) maps cleanly to the B.A.D.D.I.E. architecture, where performance-critical operations (inference, audio processing) run natively while the UI remains responsive.

5.5 Performance Characteristics

The current prototype achieves the following performance characteristics on the development environment:

Table 5: Measured performance characteristics

Metric	Value
UI render time	< 16 ms (60 fps)
Memory query latency	< 1 ms (in-memory store)
Token streaming latency	Network-dependent (Ollama)
Build time (frontend)	3.33 s
Build time (Rust backend)	~45 s (cold)
Bundle size (frontend)	~330 kB

The Rust backend build time of approximately 45 seconds (cold build) is typical for Tauri applications and can be reduced to under 10 seconds with incremental compilation. The frontend bundle at 330 kB is lightweight and loads instantly within the Tauri WebView.

6 DISCUSSION

6.1 Behavioural Implications

The “baddie” personality model—characterized by confidence, wit, and emotional intelligence—represents a deliberate design choice to create a companion that feels engaging and entertaining rather than purely utilitarian. This aligns with findings that personality-rich AI companions generate stronger user engagement and satisfaction [2].

However, the INTIMA benchmark findings [9] caution that companionship-reinforcing behaviours can become problematic when users express vulnerability. B.A.D.D.I.E. addresses this through its boundary-maintaining layer, which detects emotionally sensitive contexts and modulates the personality to be more supportive and less performatively “baddie.”

6.2 Privacy and Ethics

The fully local architecture is an ethical design decision. By keeping all data on-device and running inference locally, B.A.D.D.I.E. eliminates the privacy risks inherent in cloud-based companions, where intimate conversations may be stored on third-party servers, used for model training, or exposed in data breaches. No API keys, no network calls for inference, no data leaves the machine.

The system also implements:

- User control: All personality parameters, memory contents, and behavioural settings are user-accessible and modifiable.
- Transparency: The companion discloses its nature as an AI system and does not attempt to deceive the user about its capabilities or limitations.
- Boundary maintenance: The system is designed to avoid fostering unhealthy dependency, following the ethical guidelines proposed by the INTIMA researchers.
- Model ownership: As an open-weight model under Apache 2.0, Gemma 4 allows users to inspect, modify, and fine-tune the model that powers their companion.

6.3 Limitations and Future Work

Current limitations include:

- Model capability: Gemma 4 E4B, while efficient, has lower absolute capability than frontier models. Future work will evaluate larger local variants (Gemma 4 31B Dense, Gemma 4 26B A4B MoE) as consumer hardware advances. The 31B variant ranks as the #3 open model on the Arena AI leaderboard.
- Personality evaluation: The personality consistency of the “baddie” persona has not been formally evaluated. We plan to adopt the CharacterBench [17] and INTIMA [9] benchmarks for systematic evaluation.
- Memory scalability: Long-term memory management over months and years of interaction remains an open challenge. Hierarchical memory compression strategies [4] will be explored.
- Multilingual support: Gemma 4 E4B supports 140+ languages, but the voice pipeline’s multilingual TTS

quality varies by language. Integration of language-specific TTS models is planned.

Future work will also explore multi-agent coordination, where B.A.D.D.I.E. can spawn specialized sub-agents for complex tasks, and integration with smart home systems for environmental control.

7 CONCLUSION

We have presented B.A.D.D.I.E., a fully local AI companion system that integrates personality modelling, long-term memory, multimodal perception, real-time voice interaction, and Live2D avatar embodiment within a unified desktop application. By combining Gemma 4 E4B’s efficient local reasoning with a typed-memory RAG system and a fully local voice pipeline, B.A.D.D.I.E. delivers a privacy-preserving, personality-rich companion experience on consumer hardware with zero cloud dependency.

The system contributes a practical architectural blueprint for researchers and developers building AI companions, and opens important discussions about the behavioural, ethical, and technical dimensions of creating digitally-defined independent entities that users can truly call their own.

As open-weight models continue to advance and consumer hardware becomes increasingly capable, systems like B.A.D.D.I.E. represent a future where AI companions are not cloud services we rent, but digital entities we own—persistent, private, and truly personal.

REFERENCES

- [1] Google AI. 2026. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core/model_card_4.
- [2] Jiangjie Chen, Xintao Wang, Rui Xu, Siyu Yuan, Yikai Zhang, Wei Shi, Jian Xie, Shuang Li, Ruihan Yang, Tinghui Zhu, Aili Chen, Nianqi Li, Lida Chen, Caiyu Hu, Siye Wu, Scott Ren, Ziquan Fu, and Yanghua Xiao. 2024. From Persona to Personalization: A Survey on Role-Playing Language Agents. *Transactions on Machine Learning Research* (2024). <https://mlanthology.org/tmlr/2024/chen2024tmlr-persona/>
- [3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Amit Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025).
- [4] Yuxuan Fang, Zhihua Pan, Zhiyuan Hu, Shengnan Xu, Yifan Liu, Yuhui Zhang, Shuchang Zhou, and Jianfeng Gao. 2026. Beyond RAG for Agent Memory: Retrieval by Decoupling and Aggregation. *arXiv preprint arXiv:2602.02007* (2026).
- [5] Google. 2026. Gemma 4: Our most capable open models to date. <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>.
- [6] HemantBK. 2026. AI-Voice-Assistant: Production-Leaning Local Voice Pipeline. <https://github.com/HemantBK/AI-Voice-Assistant>.
- [7] InsiderLLM. 2026. Talk to Your Local LLM: Voice Chat Setup. <https://www.insiderllm.com/guides/voice-chat-local-llms-whisper-tts/>.
- [8] JRM-FusionAL. 2026. Christopher—Local Voice AI Assistant. <https://github.com/JRM-FusionAL/Christopher-AI>.
- [9] Hyunji Lee, Seongjin Park, Jihye Kim, Yuna Choi, Seungjin Park, Minji Kim, Jihoon Lee, Jaehyuk Park, and Sungroh Yoon. 2025. INTIMA: A Benchmark for Human-AI Companionship Behavior. *arXiv preprint arXiv:2508.09998* (2025).
- [10] LiliLk. 2025. Local-AI-Companion: Fully Offline Voice AI with Live2D Avatar. <https://github.com/LiliLk/Local-AI-Companion>.
- [11] Aske Plaat, Ahmed Abdalrahem, and Maarten van der Meer. 2025. ENGRAM: Effective, Lightweight Memory Orchestration for Conversational Agents. *arXiv preprint arXiv:2511.12960* (2025).

- [12] Rasmus Rasmussen et al. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint arXiv:2501.13956 (2025).
- [13] Amine Tissaoui, Tao Men, Weijie Liu, Jingjing Zhang, Xing Li, Xinyu Lin, Dianbo Sui, Yang Cao, Kai Liu, and Jing Zhao. 2026. From Prompt to Persona: A Literature Review on LLMs as Single Cognitive Agents. *Journal of Ambient Intelligence and Humanized Computing* 17 (2026), 205–221. <https://doi.org/10.1007/s12652-025-05029-4>
- [14] Yu-Min Tseng, Pei-Shan Chi, Ding-Chieh Huang, et al. 2024. Two Tales of Persona in LLMs: A Survey of Role-Playing and Personalization. arXiv preprint arXiv:2406.01171 (2024).
- [15] Yuxuan Wang, Xiaoyu Liang, Mengyuan Tao, Yuxi Xia, Jing Wang, Kaiqiang Li, Yuxuan Wang, Jingjing Yang, Tangming Shi, Meng Zhang, and Xingxing Wang. 2025. Modeling, Evaluating, and Embodying Personality in LLMs: A Survey. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. <https://aclanthology.org/2025.findings-emnlp.506.pdf>
- [16] Joseph Weizenbaum. 1966. ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine. *Commun. ACM* 9, 1 (1966), 36–45. <https://doi.org/10.1145/365153.365168>
- [17] Andy Zhou, Jing Tang, et al. 2024. CharacterBench: Benchmarking Persona Consistency in LLM Agents. arXiv preprint arXiv:2403.08888 (2024).